EXERCICIO C

Aluno:Ruan Silva Carvalho

Exercício C1 - Largura de Banda da Luz

```
#include <stdio.h>

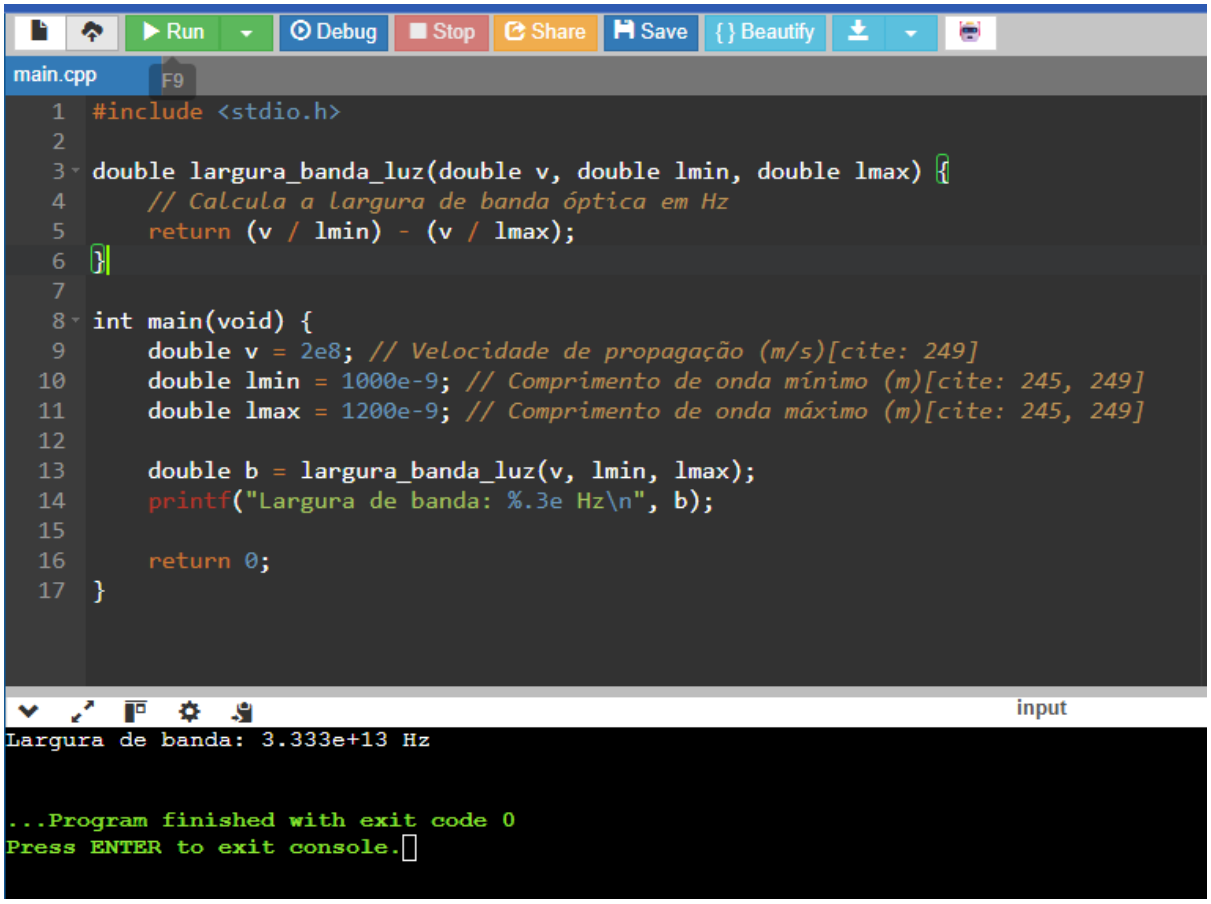

double largura_banda_luz(double v, double lmin, double lmax) {
   // Calcula a largura de banda óptica em Hz
   return (v / lmin) - (v / lmax);
}


int main(void) {
   double v = 2e8; // Velocidade de propagação (m/s)[cite: 249]
   double lmin = 1000e-9; // Comprimento de onda mínimo (m)[cite: 245, 249]
   double lmax = 1200e-9; // Comprimento de onda máximo (m)[cite: 245, 249]


   double b = largura_banda_luz(v, lmin, lmax);
   printf("Largura de banda: %.3e Hz\n", b);


   return 0;
}
```

Exercício C1: Este calcula a **largura de banda da luz** para uma dada faixa de comprimentos de onda numa fibra óptica. Basicamente, divide a velocidade de propagação pelos comprimentos de onda mínimo e máximo e subtrai os valores para encontrar o intervalo em Hz



Exercício C2 - Retardo de Propagação

```
#include <stdio.h>


double retardo(double distancia_m, double v) {
   return distancia_m / v;
}


int main(void) {
   double distancia, v = 2e8; // Velocidade padrão de 2*10^8 m/s[cite: 251]

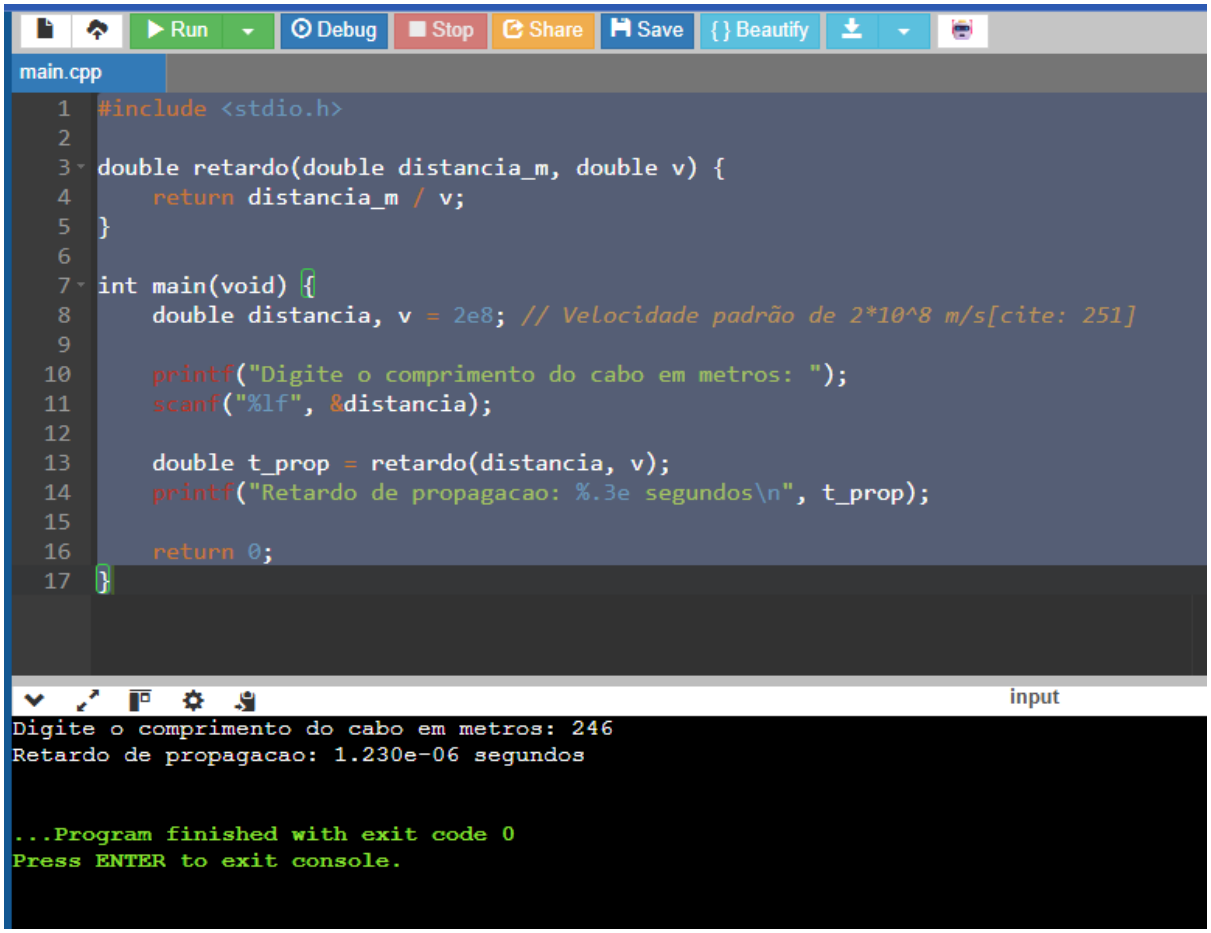

   printf("Digite o comprimento do cabo em metros: ");
   scanf("%lf", &distancia);


   double t_prop = retardo(distancia, v);
```

```
    printf("Retardo de propagacao: %.3e segundos\n", t_prop);


    return 0;
}
```

Exercício C2: Este calcula o **retardo de propagação** de um sinal luminoso. Pega no comprimento total do cabo em metros e divide-o pela velocidade de propagação da luz no meio (normalmente $2 \times 10^8\text{ m/s}$) para descobrir quanto tempo demora a chegar ao destino.



Exercício C3 - Refração ou Reflexão

```
#include <stdio.h>

#include <math.h>


const char *classifica(double incidencia, double critico) {

    if (fabs(incidencia - critico) < 1e-9)
```

```
    return "Refracao ao longo da interface";
  if (incidencia < critico)
    return "Refracao";
  return "Reflexao";
}

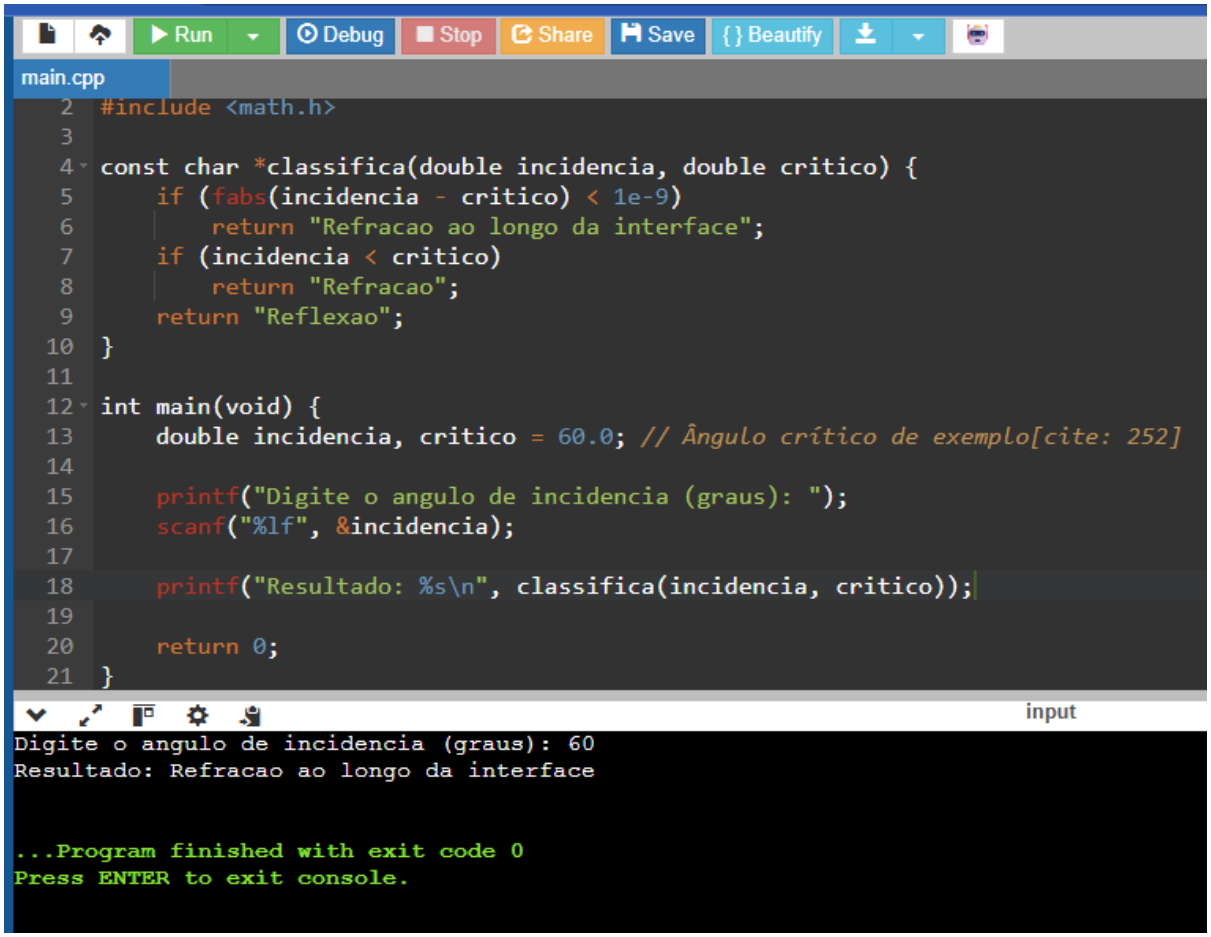
int main(void) {
  double incidencia, critico = 60.0; // Ângulo crítico de exemplo[cite: 252]

  printf("Digite o angulo de incidencia (graus): ");
  scanf("%lf", &incidencia);

  printf("Resultado: %s\n", classifica(incidencia, critico));

  return 0;
}
```

Exercício C3: Este ajuda a prever se um feixe de luz sofre **refração ou reflexão**. Compara o ângulo de incidência com o ângulo crítico do meio: se for menor, dá refração; se for igual, reflete-se ao longo da interface; e se for maior, acontece a reflexão total que mantém a luz presa na fibra



Exercício C4 - Potência no Final de um Cabo

```
#include <stdio.h>

#include <math.h>


int main(void) {

   double p_inicial, atenuacao_db_km, comprimento_km;


   printf("Digite a potencia inicial (mW): ");

   scanf("%lf", &p_inicial);


   printf("Digite a atenuacao do cabo (dB/km): ");

   scanf("%lf", &atenuacao_db_km);
```

```
    printf("Digite o comprimento do cabo (km): ");
    scanf("%lf", &comprimento_km);

    double atenuacao_total = atenuacao_db_km * comprimento_km;
    double p_final = p_inicial * pow(10.0, -atenuacao_total / 10.0);

    printf("Potencia final do sinal: %.6f mW\n", p_final);

    return 0;
}
```

Exercício C4: Este calcula a **potência final de um sinal** no fim de um cabo. Multiplica a atenuação por quilômetro pelo comprimento do cabo para achar a perda total em dB, e depois usa uma função matemática de potência de base 10 (pow) para ver quanta energia do sinal sobrou em mW.

main.cpp

```
3
4  int main(void) {
5      double p_inicial, atenuacao_db_km, comprimento_km;
6
7      printf("Digite a potencia inicial (mW): ");
8      scanf("%lf", &p_inicial);
9
10     printf("Digite a atenuacao do cabo (dB/km): ");
11     scanf("%lf", &atenuacao_db_km);
12
13     printf("Digite o comprimento do cabo (km): ");
14     scanf("%lf", &comprimento_km);
15
16     double atenuacao_total = atenuacao_db_km * comprimento_km;
17     double p_final = p_inicial * pow(10.0, -atenuacao_total / 10.0);
18
19     printf("Potencia final do sinal: %.6f mW\n", p_final);
20
21     return 0;
22 }
```

input

```
Digite a potencia inicial (mW): 56
Digite a atenuacao do cabo (dB/km): 45
Digite o comprimento do cabo (km): 34
Potencia final do sinal: 0.000000 mW


...Program finished with exit code 0
Press ENTER to exit console.
```

Exercício C5 - Categoria Mínima de Cabo UTP

```
#include <stdio.h>

typedef struct {
   int categoria;
   double taxa_max_mbps;
} CategoriaUTP;

int main(void) {
   // Tabela simplificada baseada nas categorias EIA (Cat 1, Cat 3, Cat 5, Cat 5E, Cat 6, Cat 7)[cite: 201, 260]
   CategoriaUTP tabela[] = {
      {1, 0.1},
      {3, 10.0},
      {5, 100.0},
      {52, 125.0}, // Cat 5E
      {6, 200.0},
      {7, 600.0}
   };
   int total = sizeof(tabela) / sizeof(tabela[0]);

   double taxa_desejada;
   printf("Digite a taxa de dados desejada (Mbps): ");
   scanf("%lf", &taxa_desejada);

   int encontrada = 0;
   for (int i = 0; i < total; i++) {
      if (tabela[i].taxa_max_mbps >= taxa_desejada) {
```

```
        printf("A menor categoria de cabo UTP recomendada e: Categoria %d (suporta ate %.1f Mbps)\n",
               tabela[i].categoria == 52 ? 5 : tabela[i].categoria, tabela[i].taxa_max_mbps);
        encontrada = 1;
        break;
    }
  }

  if (!encontrada) {
    printf("Taxa muito alta para as categorias padrao cadastradas.\n");
  }

  return 0;
}
```

Exercício C5: Este descobre a **categoria mínima de um cabo UTP**. Com base numa tabela com as normas EIA e as respetivas taxas máximas (como Cat 3, Cat 5, Cat 5E, Cat 6, Cat 7), o programa lê a taxa que pretendemos em Mbps e diz-nos qual é o cabo mais económico ou adequado que suporta essa velocidade.

